

Reporte técnico – Ensamblador y enlazador

IA-32

Materia: Estructura y Programación de Computadoras (2026-2)

Profesora: Ing. Adara Mercado Martínez

Equipo:

Dávalos Carmona Gael Eduardo

Morales Basilio Alejandra Sofía

Reyes García Miguel Ángel

Santos Cruz Lair Abraham

Torres Rodríguez Lizeth Danae

1. Introducción

El proyecto implementa, en lenguaje C, un **ensamblador** para un subconjunto de la arquitectura **IA-32** en dos variantes (una y dos pasadas) y un **mini enlazador** que combina varios módulos objeto en una imagen final.

El objetivo no es competir con `nasm` o `ld`, sino **entender y construir** las piezas centrales de una cadena de ensamblado: análisis léxico y sintáctico manuales, generación de código máquina, tabla de símbolos, formato objeto, relocalaciones y resolución de símbolos entre módulos.

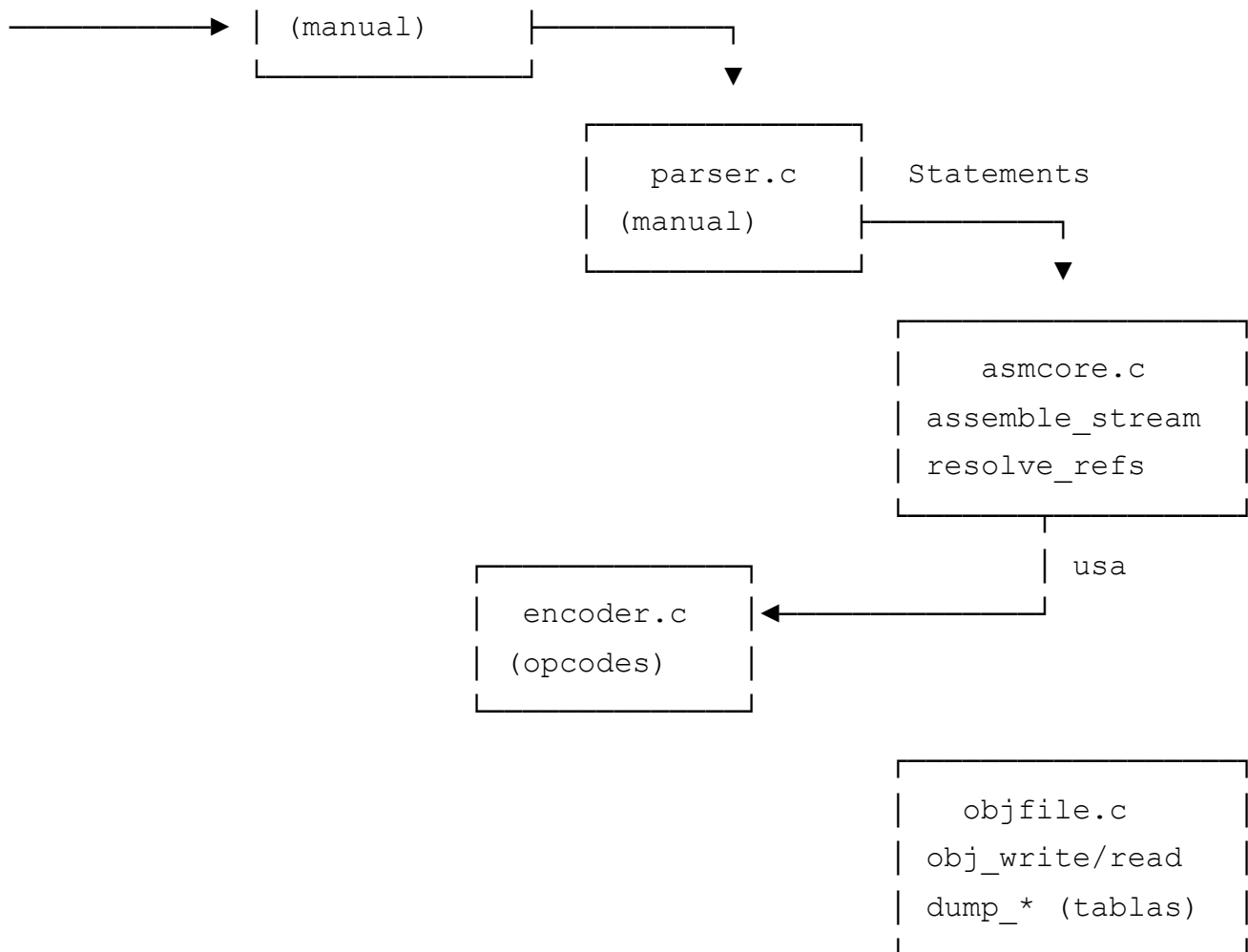
Restricciones respetadas (según el documento de la práctica):

- Analizador léxico y sintáctico **manuales** (sin `lex` / `yacc` / `flex` / `bison`).
 - Sin ensambladores ni enlazadores externos.
 - Lectura **token por token**.
-

2. Arquitectura general

El sistema está dividido en módulos con responsabilidades bien delimitadas. Los tres ejecutables comparten el mismo *backend*.

```
archivo .asm | [ lexer.c ] | tokens
```



assembler1.c ┌
 assembler2.c ┤ ──▶ usan asmcore + objfile + symtab + util
 minilinker.c └ (este último solo lee .o y reubica)

2.1. Módulos

Archivo	Responsabilidad
<code>types.c/.h</code>	Tipos base: registros, secciones, clases de símbolo, tipos de reloc.
<code>util.c/.h</code>	Utilidades: manejo de errores, memoria, lectura de archivo, <code>ByteBuf</code> .
<code>lexer.c/.h</code>	Analizador léxico manual (carácter por carácter).
<code>parser.c/.h</code>	Analizador sintáctico manual; produce una <code>Statement</code> por línea.
<code>symtab.c/.h</code>	Tabla de símbolos (arreglo dinámico): definir, declarar, buscar.
<code>encoder.c/.h</code>	Generación de código máquina IA-32 (opcode/ModRM/SIB/disp/imm).
<code>objfile.c/.h</code>	Escritura/lectura del <code>.o</code> y volcado de las tablas <code>.txt</code> .
<code>asmcore.c/.h</code>	Núcleo común de ensamblado y resolución de referencias.
<code>assembler1.c</code>	<code>main</code> del ensamblador de una pasada.

Archivo	Responsabilidad
<code>assembler2.c</code>	<code>main</code> del ensamblador de dos pasadas.
<code>minilinker.c</code>	<code>main</code> del enlazador.

3. Análisis léxico (`lexer.c`)

El lexer recorre el texto **carácter por carácter** y entrega un token a la vez. Reconoce:

- **Registros** (se etiquetan con un tipo propio, `T_REGISTER`),
- **Identificadores** (etiquetas, mnemónicos, nombres de sección),
- **Números** decimales y hexadecimales (`0x...`),
- **Cadenas** entre comillas dobles,
- **Puntuación**: `,` `[` `]` `+` `-` `*` `:` `,`
- **Salto de línea** (significativos, separan instrucciones),
- **Comentarios** con `;` (se descartan).

No se usa ninguna herramienta generadora; el control de estados es explícito.

4. Análisis sintáctico (`parser.c`)

El parser agrupa los tokens de cada línea lógica y construye una `Statement` , que puede ser:

- Una **etiqueta** (`etiqueta:`), opcionalmente seguida de instrucción/directiva,
- Una **directiva** (`section` , `global` , `extern` , `db/dw/dd` , `resb/resw/resd` , `equ` , `org`),
- Una **instrucción** con sus operandos.

El reto principal está en los **operandos de memoria**. Se implementó una

rutina (`parse_mem`) que interpreta expresiones del tipo

`[ base + indice*escala + desplazamiento ]` recorriendo términos separados por `+` / `-` , distinguiendo:

- Base simple,
- Índice escalado (`registro * escala`),
- Desplazamiento numérico,
- Símbolo (dirección).

Se validan las reglas de IA-32: el índice no puede ser `ESP` , y la escala debe ser 1, 2, 4 u 8.

5. Tabla de símbolos (`syntab.c`)

Arreglo dinámico de entradas `Symbol`. Cada símbolo guarda nombre, **clase** (`LABEL`, `CONST`, `EXTERN`, indefinido), sección, **valor/offset**, si es **global** y si está **definido**.

Operaciones:

- `define_label` — Define una etiqueta; marca **redefinición** si ya estaba definida.
- `define_const` — Define una constante `equ`.
- `declare_extern` — Declara un símbolo externo (sin sobrescribir definiciones locales).
- `mark_global` — Marca un símbolo como exportable.
- `find` — Búsqueda por nombre.

La detección de redefiniciones y el caso "externo definido localmente" se manejan aquí y en `asmcore`.

6. Generación de código (`encoder.c`)

El encoder es **puramente sintáctico**: dada una instrucción, produce los bytes y una lista de *referencias* (campos de 32 bits que dependen de un símbolo). No consulta la tabla de símbolos, lo que mantiene el diseño desacoplado.

Punto de diseño importante: **el tamaño de cada instrucción depende solo de la forma de sus operandos** (no de si un símbolo está o no resuelto). Para lograrlo los saltos siempre son cercanos (`rel32`). Gracias a esto, la **pasada 1** del ensamblador de dos pasadas calcula exactamente los mismos tamaños que la **pasada 2**, y el ensamblador de una pasada nunca tiene que "reescribir" instrucciones.

Las decisiones detalladas de opcodes, ModRM y SIB están en [formato_objeto.md](#), sección 4.

7. Núcleo de ensamblado (`asmcore.c`)

Ambos ensambladores usan la misma función `assemble_stream`, parametrizada por `define_labels`:

- Con `define_labels = 1` (una pasada y pasada 1 de dos pasadas): Define etiquetas, constantes, externos y globales.

- Con `define_labels = 0` (pasada 2): **No** redefine símbolos, pero sigue cambiando de sección y emitiendo bytes/datos.

En ambos casos se emiten los bytes y se registran las **referencias**

pendientes (fixups). Después, `resolve_refs` convierte cada referencia en:

- Un **parche local** (salto relativo dentro de la misma sección, o constante `equ`), o
- Una **relocación** (`ABS32`, o `REL32` entre secciones, o cualquier referencia a un `extern`).

Esto centraliza toda la lógica delicada en un solo lugar y garantiza que los dos ensambladores se comporten igual.

8. Formato objeto y tablas (`objfile.c`)

El `.o` es de **texto** (ver [formato_objeto.md](#)). Incluye encabezado, tabla de secciones, tabla de símbolos, tabla de relocalaciones y los bytes de `.text` / `.data` en hexadecimal. `objfile.c` también genera las tablas `.txt` legibles que pide la práctica (símbolos, secciones, relocalaciones, referencias y tamaños).

9. Enlazador (`minilinker.c`)

Pasos:

1. Lee N módulos objeto.
2. Calcula el **tamaño total** de cada sección y las **bases globales**: `.text` en `0x00000000`, luego `.data` y `.bss` alineadas a 4 bytes.
3. Calcula la base de cada par (módulo, sección) según el orden de los módulos.
4. **Fusiona** los buffers de `.text` y de `.data`.
5. Construye el **mapa de símbolos globales** y detecta duplicados.
6. Para cada relocación: resuelve el símbolo (primero local del módulo, luego global), calcula el valor según `ABS32` / `REL32` y **parcha** el buffer.
7. Escribe la imagen final en hexadecimal y el mapa de símbolos.

Errores que detecta: símbolo global duplicado, símbolo externo no resuelto y relocación fuera de rango.

10. Pruebas y resultados

El script `scripts/run_tests.sh` ejecuta todos los casos. Casos cubiertos (los mínimos exigidos por la práctica):

Caso	Archivo	Qué demuestra
MOV inmediato	<code>tests/test_mov_imm.asm</code>	inmediatos dec/hex/negativos
Salto	<code>tests/test_jumps.asm</code>	condicionales e incondicional
Referencias adelant.	<code>tests/test_forward.asm</code>	forward refs + fixups
EXTERN	<code>tests/test_extern.asm</code>	relocaciones a externos
CALL	<code>tests/test_call.asm</code>	llamada a etiqueta local
Relocaciones	<code>tests/test_reloc.asm</code>	ABS32 a datos y dd símbolo
SIB	<code>tests/test_sib.asm</code>	todos los modos de direccionamiento
Múltiples módulos	<code>examples/mod1/mod2.asm</code>	enlace y resolución de símbolos

Ejemplo verificado de codificación (`mov ebx, eax`): el opcode `8B` corresponde a `mov r32, r/m32`; el byte ModRM resulta `0xD8` = `mod=11`, `reg=011` (EBX), `rm=000` (EAX), es decir, registro a registro. El resultado coincide con el que produce un ensamblador estándar.

11. División del trabajo

Integrante	Responsabilidad principal
Gael	Parser y Representación interna
Sofía	Lexer y tokens (análisis léxico/sintáctico) y casos de prueba.
Ángel	Encoder (opcodes, ModRM/SIB) y núcleo <code>asmcore</code> .
Lair	Enlazador, scripts de build/pruebas y documentación.
Lizeth	Tabla de símbolos, formato objeto y volcado de tablas.

12. Mapeo con la rúbrica de evaluación

Criterio	Dónde se cumple
Análisis léxico/sintáctico manual	<code>lexer.c</code> , <code>parser.c</code>
Modos de direccionamiento (incl. SIB)	<code>parser.c</code> (<code>parse_mem</code>), <code>encoder.c</code>

Criterio	Dónde se cumple
Ensamblador de una pasada	<code>assembler1.c + asmcore.c</code>
Ensamblador de dos pasadas	<code>assembler2.c + asmcore.c</code>
Formato objeto con tablas	<code>objfile.c</code> , <code>formato_objeto.md</code>
Relocaciones	<code>asmcore.c (resolve_refs)</code> , <code>minilinker.c</code>
Enlazador de múltiples módulos	<code>minilinker.c</code>
Manejo de errores	<code>util.c (error_at)</code> usado en todo el código
Documentación	carpeta <code>docs/</code>
Casos de prueba	<code>tests/</code> , <code>examples/</code> , <code>scripts/run_tests.sh</code>

13. Posibles extensiones

- Saltos cortos (`rel8`) con selección automática de tamaño.
- Soporte completo de operandos de 8/16 bits y más formas de `movzx` / `movsx` .
- Salida binaria real además del volcado hexadecimal.
- Uso efectivo de `org` para colocar secciones en direcciones fijas.